AI-POWERED TRAFFIC CONGESTION CLASSIFICATION USING CNN TRANSFER LEARNING (MOBILENETV2)

Course: Neural Networks and Deep Learning

Submitted by: Essis Desiree

UID: 25MDA10002

GitHub Repository: https://github.com/EssisDez/ai-traffic-congestion-classification.git

Abstract

Traffic congestion causes lost time, higher fuel use and increased emissions. This project builds an image-based classifier that labels a road scene as Low, Moderate or High congestion. A MobileNetV2 network pre-trained on ImageNet is adapted through transfer learning and then fine-tuned on a custom traffic image dataset. The trained model is deployed through a Gradio web interface where a user uploads an image and instantly sees the predicted congestion level with class probabilities. On the validation set (240 images) the final model reached an accuracy of 71.25%, with high recall for the Low and High classes and weaker recall for the Moderate class.

1. Introduction

Urban road networks are monitored with thousands of cameras, but most footage is still reviewed manually. Automatically converting a camera frame into a simple congestion level can support signal timing, route advice and city planning. Deep convolutional neural networks (CNNs) are well suited to this task because they learn visual features such as vehicle density and spacing directly from pixels.

Training a CNN from scratch needs very large datasets. Transfer learning avoids this by reusing features learned on a large source dataset (ImageNet) and adapting them to a smaller target task. This project applies that idea to three-class traffic congestion classification.

2. Objectives

1. Build a three-class image classifier (Low, Moderate, High congestion).
2. Use MobileNetV2 with transfer learning, followed by fine-tuning of the top layers.
3. Evaluate the model with accuracy, a classification report and a confusion matrix.
4. Provide an interactive Gradio interface for uploading an image and viewing the prediction.
5. Publish the code and documentation in a public GitHub repository.

3. Dataset

The dataset is organised in three folders named low, moderate and high, one per congestion class. Images are loaded with Keras image_dataset_from_directory, resized to 224 x 224 pixels and split 80% training (960 images) and 20% validation (240 images) using a fixed random seed (42) for reproducibility.

Class	Number of images
Low	400 (of 4,331 available)
Moderate	400 (of 1,187 available)
High	400 (of 585 available)
Total	1,200

Dataset source: the public Hugging Face dataset “traffic-camera-norway-images” (ilsilfverskiold/traffic-camera-norway-images, https://huggingface.co/datasets/ilsilfverskiold/traffic-camera-norway-images), which contains road traffic camera images from Norway. Its original labels (high-, medium-, low- and no-traffic) were mapped to the three project classes: medium-traffic became Moderate, and no-traffic images were merged into Low. The training split used here holds 6,103 images (4,331 Low including no-traffic, 1,187 Moderate and 585 High). Because Low is far more frequent, 400 images were randomly selected from each class (seed 42), giving a balanced working set of 1,200 images. The dataset page does not state a licence, so the images are used here only for academic, non-commercial purposes and are credited to the dataset author.

4. Methodology

4.1 Pre-processing and augmentation

All images are resized to 224 x 224 and pixel values are rescaled to the range [-1, 1], which is the input range MobileNetV2 expects. To reduce overfitting, three random augmentations are applied during training only: horizontal flip, rotation of up to 5% of a full turn, and zoom of up to 10%.

4.2 Model architecture

The model uses MobileNetV2 (ImageNet weights, without the original classification head) as a feature extractor. On top of it sit a global average pooling layer, a dropout layer (rate 0.3) and a dense softmax layer with three outputs. MobileNetV2 was chosen because it is lightweight and fast, which suits a real-time traffic-monitoring scenario.

4.3 Training strategy

Stage 1 – Transfer learning. The MobileNetV2 base is frozen and only the new classification head is trained. Optimiser: Adam, learning rate 0.001; loss: sparse categorical cross-entropy; up to 10 epochs.

Stage 2 – Fine-tuning. The last 30 layers of the base network are unfrozen while Batch Normalization layers stay frozen for stability. The model is recompiled with a much smaller learning rate (1e-5) and trained for up to 10 further epochs.

Both stages use early stopping on validation loss (patience 3, best weights restored) and a checkpoint that saves the best model.

5. Implementation

Component	Tool
Language	Python 3
Deep learning	TensorFlow / Keras
Evaluation	scikit-learn, Matplotlib
Interface	Gradio
Training environment	Google Colab

The Gradio interface accepts an uploaded image, converts it to RGB, resizes it to 224 x 224, passes it to the trained model and displays the probability of each congestion class. The interface is also packaged as a standalone `app.py` so it can be run locally or deployed.

6. Results

6.1 Overall performance

Metric	Value
Validation accuracy	71.25%
Validation loss	0.6478
Accuracy after Stage 1 only	66.25% (best epoch; early stopping ended Stage 1 after 9 epochs)
Accuracy after fine-tuning	71.25% (peak 73.75% during fine-tuning)

6.2 Per-class performance

Class	Precision	Recall	F1-score
Low	0.73	0.82	0.77
Moderate	0.73	0.43	0.54
High	0.69	0.90	0.78

6.3 Figures

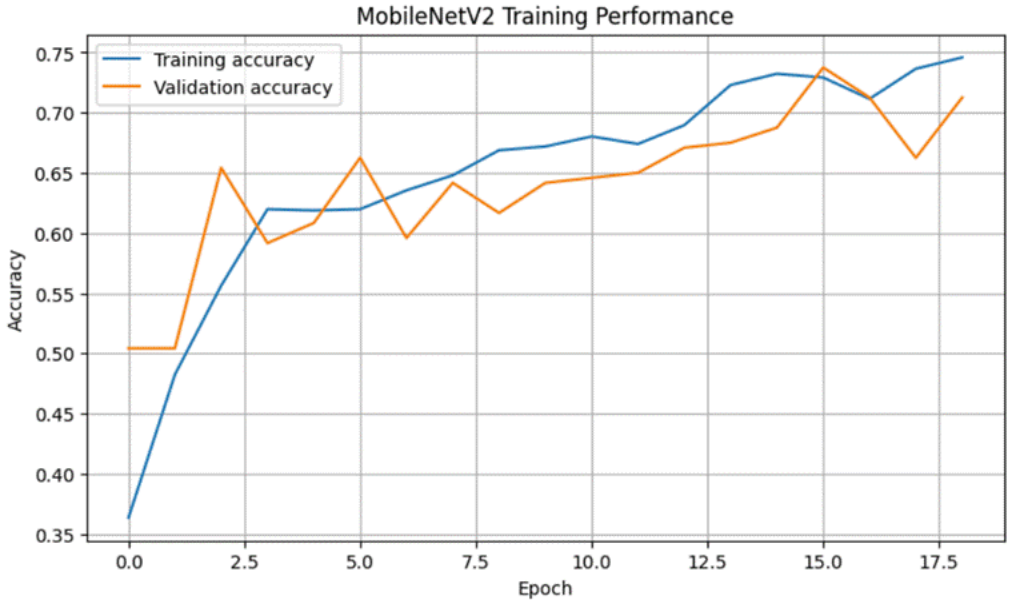


Figure 1: Training and validation accuracy over 19 epochs (Stage 1: epochs 0 to 8; fine-tuning: epochs 9 to 18).

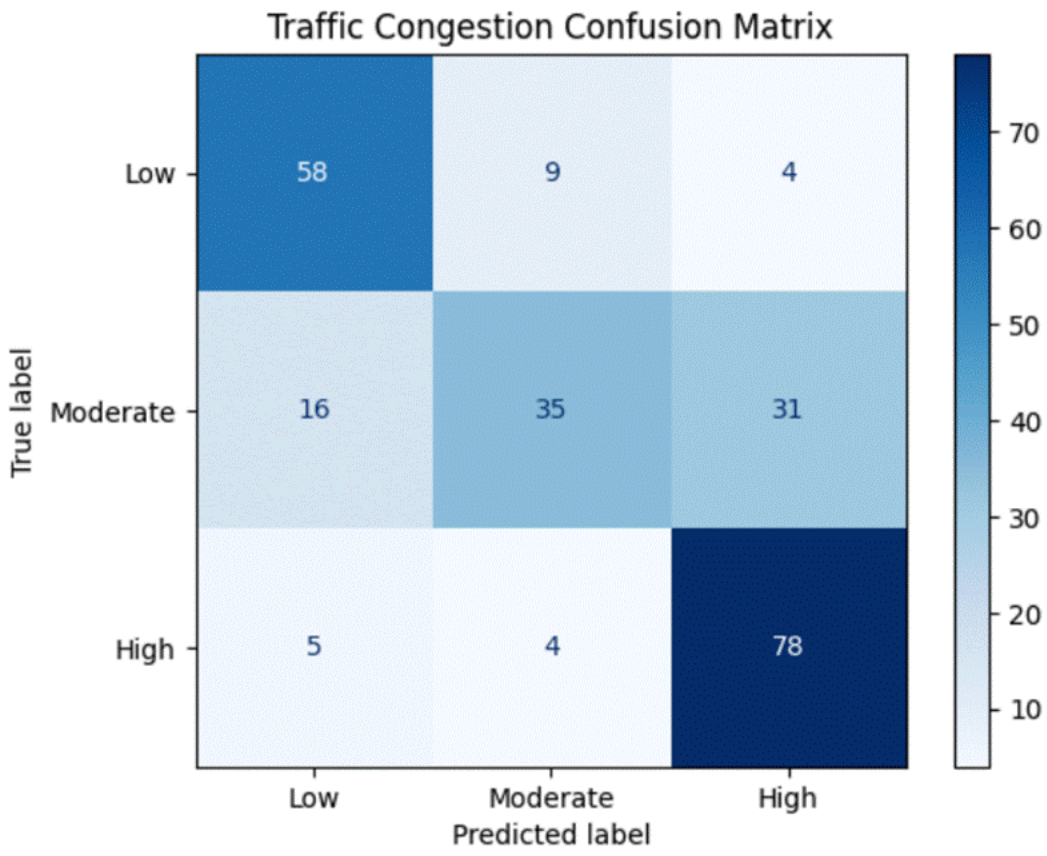


Figure 2: Confusion matrix on the 240 validation images (rows: true class, columns: predicted class).

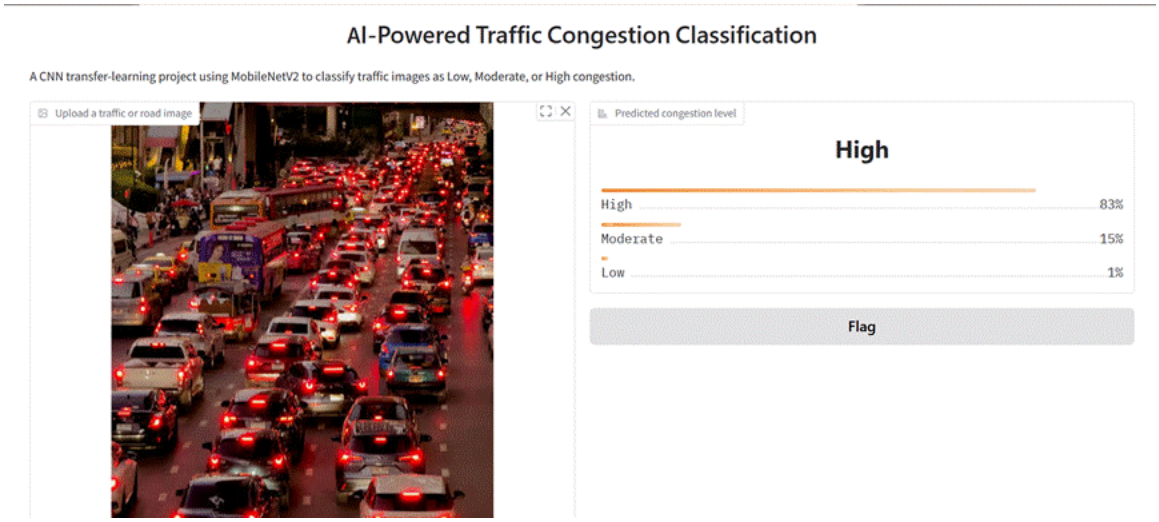


Figure 3: The Gradio interface after uploading a congested road image. The model predicts High congestion (83%), with Moderate at 15% and Low at 1%.

6.4 Discussion

The final model reached 71.25% validation accuracy with a validation loss of 0.648. Fine-tuning helped: accuracy rose from 66.25% after Stage 1 to 71.25% once the last 30 layers were adapted to traffic scenes, and the validation loss fell from 0.76 to 0.65. High congestion was the easiest class to detect (recall 0.90, F1 0.78), followed by Low (recall 0.82, F1 0.77). Moderate was clearly the hardest, with a recall of only 0.43 and an F1-score of 0.54: of the 82 Moderate validation images, 35 were classified correctly, 31 were predicted as High and 16 as Low. This is expected, because Moderate lies between the other two classes and the boundary is partly subjective, so errors cluster around it. Training accuracy (about 75%) stayed close to validation accuracy (about 71%), so the model is not strongly overfitted; the remaining error points to limited data and noisy labels rather than memorisation. Because the validation set has only 240 images, one image changes the accuracy by about 0.4 percentage points, and the validation curve fluctuates from epoch to epoch (a peak of 73.75% before settling at 71.25%), so the result should be read as approximate.

7. Limitations

- The validation set was used both to choose the best epoch and to report final accuracy, so the reported accuracy may be slightly optimistic. A separate untouched test set would give a more reliable estimate.
- Congestion levels are subjective, so labels at the Low/Moderate and Moderate/High boundaries can be inconsistent.

- The images come from fixed road cameras, so many frames look very similar and near-duplicates may appear in both the training and validation splits, which can make the accuracy look better than it would on new cameras.
- Only 400 images per class (1,200 in total) were used to keep the classes balanced, so most of the available Low images were left unused, and performance on other lighting, weather, camera angles or countries has not been tested.

8. Conclusion and Future Work

This project showed that a lightweight MobileNetV2 model, adapted through transfer learning and fine-tuning, can classify traffic images into three congestion levels (71.25% validation accuracy) and can be delivered to non-technical users through a simple web interface. Future work includes improving recall for the Moderate class with more and better-labelled images, a separate test set, class-weighting or more data for under-represented classes, comparison with other backbones such as EfficientNet or ResNet, applying the model to live video streams, and deployment on a public platform such as Hugging Face Spaces.

9. Code Availability

The complete source code, trained-model instructions, requirements and README are available at: https://github.com/EssisDez/ai-traffic-congestion-classification.git

Repository contents: training notebook, trained model and class_names.json, app.py (Gradio app), requirements.txt, README.md.

References

1. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. CVPR 2018.
2. Deng, J. et al. (2009). ImageNet: A Large-Scale Hierarchical Image Database. CVPR 2009.
3. Yosinski, J., Clune, J., Bengio, Y., Lipson, H. (2014). How Transferable Are Features in Deep Neural Networks? NeurIPS 2014.
4. Abadi, M. et al. (2015). TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems.
5. Abid, A. et al. (2019). Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild. arXiv:1906.02569.